```
import os
os.environ["OPENAI_API_KEY"] = "sk-proj-ORV1HwUGs8R8vWlaRYLdAzJfhxLH9NYWyb5GDGG
```

```
!pip install -q youtube-transcript-api langchain-community langchain-openai fai
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 485.2/485.2 kB 3.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.4/2.4 MB 19.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 120.4/120.4 kB 5.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.5/18.5 MB 42.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 26.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 73.1/73.1 kB 3.9 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the pa
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.34.2 which
```

```
from youtube_transcript_api import YouTubeTranscriptApi, TranscriptsDisabled
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings,ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import PromptTemplate
```

Step1 a- INDEXING(Document Ingestion)

```
 video_id = "Gfr50f6ZBvo" # only the ID, not full URL
try:
    # If you don't care which language, this returns the "best" one
    transcript_list = YouTubeTranscriptApi.get_transcript(video_id, languages=[

    # Flatten it to plain text
    transcript = " ".join(chunk["text"] for chunk in transcript_list)
    print(transcript)

except TranscriptsDisabled:
    print("No captions available for this video.")
```

Step1 b- INDEXING(TextSplitting)

```
splitter = RecursiveCharacterTextSplitter(chunk_size =1000, chunk_overlap =200)
chunks = splitter.create_documents([transcript])
```

Step1 - c INDEXING(Embedding Generation)

```
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
```

Step1 - d INDEXING(Storing in VectorStore)

```
vector_store = vector_store.FAISS(chunks,embeddings)

vector_store.index_to_docstore_id
```

```
vector_store.get_by_ids['']
```

Step2 - RETRIEVAL

```
from re import search
retriever = vector_store.as_retriever(search_type = "similarity",search_kwargs
```

```
retriever.invoke('What is Deepmind?')
```

Step3 - AUGMENTATION

```
llm = ChatOpenAI(model_name = "gpt-3.5-turbo",temperature = 0.2)
```

```
prompt = PromptTemplate(
    template = """  You are a helpful assistant.
      Answer ONLY from the provided transcript context.
      If the context is insufficient, just say you don't know.

      {context}
      Question: {question}""",
    input_variables = ["context","question"]
)
```

```
question =  "is the topic of nuclear fusion discussed in this video? if yes the
retriever_docs = retriever.invoke(question)
```

```
context_text  = "'\n\n".join(doc.page_content for doc in retriever_docs)
context_text
```

```
final_prompt  =prompt.invoke({"context":context_text,"question":question})
final_prompt
```

Step4 - GENERATION

```
answer = llm.invoke(final_prompt)
answer.content
```

BUILDING USING CHAIN

```
from langchain_core.runnables import RunnablePassthrough,RunnableLambda,RunnableI
from langchain_core.output_parsers import StrOutputParser
```

```
def format_docs(retrieved_docs):
  return "\n\n".join(doc.page_content for doc in retrieved_docs)
```

```
parallel_chain = RunnableParallel(
    'context': retriever | RunnableLambda(format_docs),
    'question':RunnablePassthrough()
)
```

```
parallel_chain.invoke('Who is Demis?')
```

```
parser = StrOutputParser()
```

```
main_chain = parallel_chain | prompt | llm | parser
```

```
main_chain.invoke("Can you summarize the video?")
```